

[총정리]

1. 셸(Shell)이란?

사용자와 커널(Kernel) 사이에서 명령어를 해석하여 전달하는 인터페이스 프로그램

2. 셸의 종류는?

본 셸(sh), C 셸(csh), 콘 셸(ksh), 배시 셸(bash), 대시 셸(dash) 등이 있고, 리눅스의 기본 셸은 주로 배시 셸(bash)이 사용됨

3. 셸 스크립트란?

셸에서 실행할 명령어들을 순서대로 기록한 텍스트 파일로 별도의 컴파일 과정 없이 실행할 수 있다.

4. 셸 변수 선언 시 주의점은?

변수명, 등호(=), 값 사이에 공백이 없어야 하며 모든 변수는 기본적으로 문자열로 취급

5. 셸 스크립트에서 숫자 계산은?

expr 키워드를 역따옴표(')로 감싸서 사용

6. 파라미터 변수란?

스크립트 실행 시 전달되는 인자(argument)들을 받는 변수

\$0은 스크립트 파일 이름, \$1은 첫 번째 인자, \$2는 두 번째 인자를 의미

7. GCC란?

GNU Compiler Collection의 약자로, C, C++, Objective-C 등 다양한 언어를 위한 컴파일러

8. GCC 컴파일 과정은?

소스 파일(.c) → 전처리기(cpp) → 컴파일러(cc1) → 어셈블리 파일(.s) → 어셈블러(as) → 오브젝트 파일(.o) → 링커(ld) → 실행 파일

9. 주요 GCC 옵션은?

-o: 출력 파일 이름을 지정

-c: 컴파일만 수행하여 오브젝트 파일(.o)을 생성

-g: 디버깅 정보를 포함

-I<경로>: 헤더 파일을 탐색할 디렉터리를 추가

-L<경로>: 라이브러리 파일을 탐색할 디렉터리를 추가

-l<라이브러리>: 특정 라이브러리를 링크

10. GDB란?

GNU 디버거(GNU Debugger)로, 프로그램의 버그를 찾기 위해 실행을 제어하고 내부 상태를 확인하는 데 사용되는 도구. 사용을 위해서는 컴파일 시 gcc -g 옵션을 추가해야 함.

11. GDB의 주요 명령어는?

break 또는 b: 중단점(breakpoint)을 설정

run 또는 r: 프로그램을 실행.

next 또는 n: 현재 함수의 다음 줄로 넘어감 (함수 안으로 들어가지 않음).

step 또는 s: 다음 줄로 넘어가되, 함수 호출 시 그 함수 안으로 들어감.

print 또는 p: 변수의 값을 출력.

continue 또는 c: 다음 중단점까지 실행을 계속.

12. 라이브러리(Library)란?

자주 사용되는 유용한 함수나 데이터들을 모아놓은 오브젝트 파일의 집합

13. 라이브러리의 종류는?

- 정적 라이브러리 (.a) : 컴파일 시 실행 파일에 라이브러리 코드가 완전히 포함되고 ar 명령으로 생성
- 공유 라이브러리 (.so) : 프로그램 실행 시 메모리에 로드되어 여러 프로그램이 공유해서 사용. gcc -shared 옵션으로 생성하며 , LD_LIBRARY_PATH 환경 변수를 통해 경로를 지정해야 사용할 수 있다.

14. make와 Makefile은?

- Make : 여러 파일로 구성된 대규모 프로그램의 빌드(컴파일 및 링크) 과정을 자동화하는 도구
- Makefile : make가 참조하는 규칙 파일로, 파일 간의 의존 관계와 빌드 명령어를 기술

15. 링크 파일의 종류는?

- 하드 링크 : 원본 파일과 동일한 I-node 번호를 공유하는 파일
- 심볼릭 링크 (소프트 링크) : 원본 파일을 가리키는 경로를 담고 있는 별개의 파일로, Windows의 '바로 가기'와 유사

16. 파일 권한을 변경하는 명령어?

chmod 명령어를 사용.

기호 모드(u+x)나 8진수 모드(755)로 권한을 변경할 수 있다.

17. 리눅스 기본 명령어

- ls : 디렉터리의 파일 및 하위 디렉터리 목록을 보여줌
- pwd : 현재 작업 중인 디렉터리의 전체 경로를 출력
- cd : 작업 디렉터를 변경
- mkdir: 새로운 디렉터를 생성
- rm: 파일이나 디렉터를 삭제. 디렉터리 삭제 시 -r 옵션을 사용
- cp : 파일이나 디렉터를 복사
- mv : 파일이나 디렉터를 이동시키거나 이름을 변경
- cat: 파일의 내용을 화면에 출력
- grep: 텍스트 파일 내에서 특정 패턴(문자열)을 검색
- find: 특정 조건에 맞는 파일을 검색
- ps: 현재 실행 중인 프로세스의 목록과 상태를 보여줌
- df: 파일 시스템별 디스크 사용량을 보여줌
- du: 특정 디렉터리나 파일이 차지하는 디스크 용량을 보여줌

18. 코드

- 셸스크립트(백업 프로그램)

```
#!/bin/bash
# 백업 프로그램
# 사용법 : ./backup.sh
echo -n '백업할 디렉터리 : '
read dir_name
if [ -d "$dir_name" ]; then
    backup_name="backup_$(date +%Y%m%d).tar.gz"
    tar -czf "$backup_name" "$dir_name"
    echo "$backup_name"
else
    echo "디렉터리가 존재하지 않습니다."
fi
```

- C언어(max, min으로 라이브러리 만들기)

```
/*main.c*/  
#include <stdio.h>  
#include "testlib.h"  
void main(){  
    printf("max(1,2) = %d\n", max(1,2));  
    printf("min(1,2) = %d\n", min(1,2));  
}
```

```
/* max.c */  
int max(int a, int b) {  
    if(a > b) return a;  
    else return b;  
}
```

```
/* min.c */  
int min(int a, int b) {  
    if(a < b) return a;  
    else return b;  
}
```

● 라이브러리 만드는 과정 : 정적, 공유라이브러리 - 각자 정리해보기(과제이기도 함)

19. Makefile 만들기

최종 실행 파일 : app

소스파일 : main.c hello.c kor.c han.c

[기본]

all : app

app : main.o hello.o kor.o han.o

gcc -o app main.o hello.o kor.o han.o

main.o : main.c

gcc -c main.c

hello.o : hello.c

gcc -c hello.c

kor.o : kor.c

gcc -c kor.c

han.o : han.c

gcc -c han.c

clean :

rm -f app main.o hello.o kor.o han.o

[변수]

CC = gcc

TARGET = app

OBJS = main.o hello.o kor.o han.o

all : \$(TARGET)

\$(TARGET) : \$(OBJS)

\$(CC) -o \$(TARGET) \$(OBJS)

main.o : main.c

\$(CC) -c main.c

hello.o : hello.c

\$(CC) -c hello.c

kor.o : kor.c

\$(CC) -c kor.c

han.o : han.c

\$(CC) -c han.c

clean :

rm -f \$(TARGET) \$(OBJS)

[최종]

CC = gcc

TARGET = app

OBJS = main.o hello.o kor.o han.o

all : \$(TARGET)

\$(TARGET) : \$(OBJS)

\$(CC) -o \$@ \$^

.c.o :

\$(CC) -c -o \$@ \$<

clean :

rm -f \$(OBJS) \$(TARGET)

[단답형 형식 복습문제]

1. 현재 작업 디렉터리의 경로를 출력하는 명령어는? → `pwd`
2. 디렉터리 목록을 표시하는 명령어는? → `ls`
3. 숨김 파일까지 포함하여 상세하게 보려면 `ls` 명령어에 어떤 옵션을 추가해야 하나? → `-la`
4. 하위 디렉터리까지 재귀적으로 표시하는 `ls` 옵션은? → `-R`
5. 디렉터를 생성하는 명령어는? → `mkdir`
6. 상위 폴더까지 동시에 생성하는 `mkdir` 옵션은? → `-p`
7. 디렉터를 삭제할 때 사용하는 명령어는? → `rm -r`
8. 강제로 삭제(확인 없이)하려면 `rm` 명령어에 어떤 옵션을 추가해야 하나? → `-f`
9. 파일을 복사하는 명령어는? → `cp`
10. 디렉터를 복사하려면 `cp` 명령어에 어떤 옵션을 추가해야 하나? → `-r`
11. 파일을 이동하거나 이름을 바꾸는 명령어는? → `mv`
12. 파일 내용을 화면에 출력하는 명령어는? → `cat`
13. 특정 문자열을 파일 내에서 찾는 명령어는? → `grep`
14. 조건에 맞는 파일을 찾는 명령어는? → `find`
15. 파일 권한을 변경하는 명령어는? → `chmod`
16. 소유자와 그룹을 변경하는 명령어는? → `chown`
17. 현재 실행 중인 프로세스를 보는 명령어는? → `ps`
18. 실시간으로 CPU, 메모리를 모니터링하는 명령어는? → `top`
19. 디스크 사용량을 확인하는 명령어는? → `df`
20. 특정 디렉터리의 용량을 확인하는 명령어는? → `du`
21. 명령어 실행 기록을 확인하는 명령어는? → `history`
22. 파일의 inode 번호를 확인하는 명령어는? → `ls -li`
23. 심볼릭 링크를 생성하는 명령어는? → `ln -s`
24. 하드 링크를 생성하는 명령어는? → `ln`
25. 셸의 역할은 사용자와 () 사이의 중계자이다. → 커널
26. 리눅스의 기본 셸은? → Bash 셸
27. 셸 스크립트 파일의 확장자는? → `.sh`
28. 현재 사용하는 셸의 종류를 확인하는 명령어는? → `echo $SHELL`
29. 환경 변수를 설정할 때 사용하는 명령어는? → `export`
30. 환경 변수를 해제할 때 사용하는 명령어는? → `unset`
31. 셸 스크립트의 첫 줄에 반드시 포함되어야 하는 해시뱅 라인은?
→ `#!/bin/sh` or `#!/bin/bash`
32. 셸 스크립트의 첫 줄에 위치하는 해시뱅(#)의 의미는? → 해당 스크립트를 어떤 셸로 실행할지 지정
33. 셸 스크립트에서 화면에 문자열을 출력하는 명령어는? → `echo`
34. 셸 스크립트에서 사용자 입력을 받는 명령어는? → `read`
35. 셸 스크립트에서 수식을 표현할 때 사용하는 명령어는? → `expr`
36. 셸 스크립트에서 조건문을 작성할 때 사용하는 구조는? → `if ... then ... fi`
38. 셸 스크립트에서 여러 조건을 비교할 때 사용하는 명령어는? → `case ... esac`
39. 셸 스크립트에서 반복문을 작성할 때 사용하는 명령어는? → `while, for`
40. 실행 권한을 주지 않으면 셸 스크립트는 어떻게 실행하는가? → `sh 파일명.sh`
41. GCC는 어떤 약자의 뜻인가? → GNU Compiler Collection
42. C언어 컴파일러 명령어는? → `gcc`
43. `gcc`에서 소스만 전처리할 때 사용하는 `gcc` 옵션은? → `-E`
44. `gcc`에서 어셈블리 파일까지만 생성하는 `gcc` 옵션은? → `-S`
45. `gcc`에서 오브젝트 파일까지만 생성하는 `gcc` 옵션은? → `-c`
46. `gcc`에서 디버깅 정보를 포함시키는 옵션은? → `-g`
47. `gcc`에서 실행 파일 이름을 지정하는 옵션은? → `-o`

48. gcc에서 헤더 파일 경로를 추가할 때 사용하는 옵션은? → -I
49. gcc에서 라이브러리 경로를 추가할 때 사용하는 옵션은? → -L
50. gcc에서 수학 라이브러리를 링크할 때 사용하는 옵션은? → -lm
51. 정적 라이브러리의 확장자는? → .a
52. 공유 라이브러리의 확장자는? → .so
53. 정적 라이브러리를 만드는 명령어는? → ar
54. ar 명령어에서 아카이브 생성 옵션은? → rcs
55. 공유 라이브러리를 만들 때 사용하는 gcc 옵션은? → -shared
56. 동적 라이브러리 로드 시 사용하는 함수는? → dlopen()
57. 심볼 주소를 찾는 함수는? → dlsym()
58. 로드된 라이브러리를 닫는 함수는? → dlclose()
59. 동적 라이브러리 사용 시 링크할 라이브러리는? → -ldl
60. 환경 변수로 공유 라이브러리 경로를 지정하는 변수는? → LD_LIBRARY_PATH
61. 여러 소스파일을 자동으로 컴파일·링크하는 도구는? → make
62. make 도구가 참조하는 파일 이름은? → Makefile
63. Makefile에서 목표(target)와 의존 파일(dependency)을 구분하는 기호는? → 콜론(:)
64. Makefile의 기본 구조는 '대상: 의존파일 → 명령'이다. 이때 각 명령은 반드시 (어디로 들여쓰기 해야 하는가?) → 탭(tab)
65. 오브젝트 파일을 정리하는 make 규칙은 보통 어떤 이름으로 지정하는가? → clean
66. make clean 명령은 어떤 동작을 하는가? → 생성된 실행·오브젝트 파일 삭제
67. 링커(linker)의 역할은? → 오브젝트 파일들을 하나의 실행 파일로 결합
68. GDB는 어떤 프로젝트에서 개발된 디버거인가? → GNU 프로젝트
69. GDB로 디버깅하려면 gcc 컴파일 시 반드시 어떤 옵션을 추가해야 하는가? → -g
70. 프로그램 실행 중 특정 위치에서 멈추게 하는 명령은? → break
71. 현재 위치의 다음 줄로 한 단계씩 실행하는 명령은? → next (n)
72. 함수 내부로 들어가면서 한 줄씩 실행하는 명령은? → step (s)
73. 중단점까지 실행을 계속하는 명령은? → continue (c)
74. 변수의 현재 값을 출력하는 명령은? → print (p)
75. 디버깅을 종료하는 명령은? → quit (q)